합성곱 신경망 (CNN)

합성곱 계층 · 풀링 계층 · im2col · SimpleConvNet · LeNet과 AlexNet

딥러닝개론및연습 · 2026년 2학기

이번 주의 핵심 질문

이번 주의 핵심 질문

- 지금까지: MNIST 28×28 이미지를 길이 784 벡터로 **펼쳐서** 입력
- 그러면 “이 픽셀의 바로 위·바로 옆에 무엇이 있는가”라는 **공간 정보**를 신경망이 직접 활용하지 못한다
- 이미지의 **모양을 유지한 채** 학습할 수는 없을까?
- 이미지의 **어느 위치에 있든** 같은 모양(예: 세로 획)을 같은 방법으로 알아보려면?

학습목표

1. 합성곱 계층이 이미지에 적합한 이유 (공간 정보, 가중치 공유)
2. 합성곱 연산의 손 계산, 패딩·스트라이드에 따른 출력 크기
3. 여러 채널·여러 필터·미니배치의 합성곱을 4차원 shape으로 설명
4. 최대 풀링의 계산과 특징
5. im2col로 합성곱을 하나의 행렬곱(Affine 계층)으로 바꾸고 역전파 유도
6. CNN으로 MNIST 학습, 학습된 필터 해석
7. LeNet과 AlexNet의 구조와 차이

목차

전체 구조

합성곱 계층

풀링 계층

합성곱 / 풀링 계층 구현하기

CNN 구현하기

CNN 시각화하기

대표적인 CNN

PyTorch로 구현하기

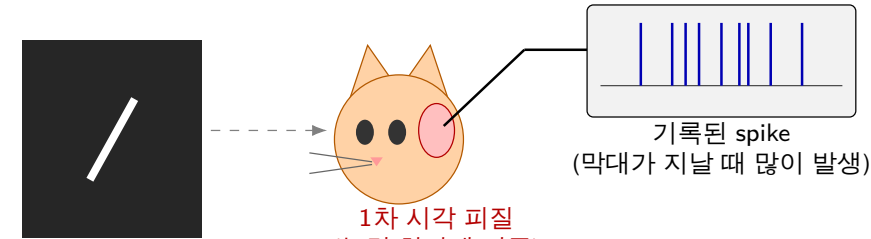
정리

연습문제

1

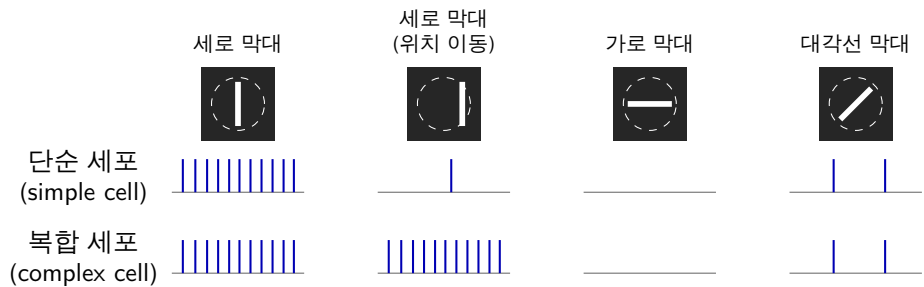
전체 구조

출발점: 고양이의 시각 피질 (Hubel과 Wiesel, 1959–1962)



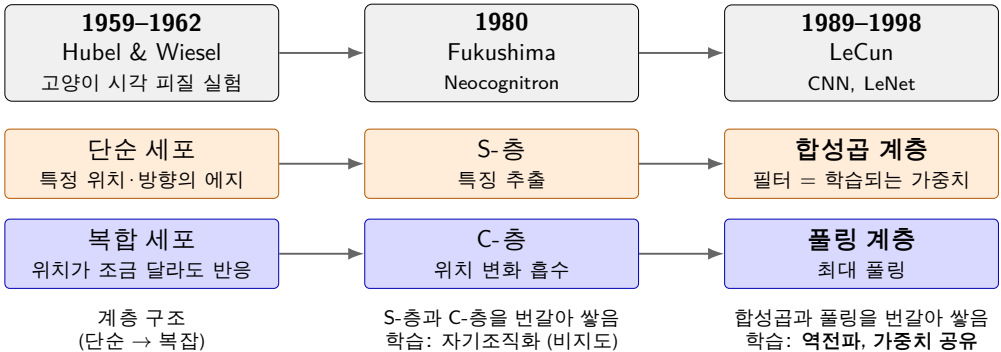
- 마취한 고양이에게 무늬를 보여 주며 **1차 시각 피질**의 뉴런 **하나**의 반응(spike)을 기록
- 점 무늬에는 거의 반응이 없다가, 슬라이드 가장자리가 만든 **비스듬한 선**이 지날 때 격렬히 반응
- CNN의 합성곱·풀링 계층은 수학이 아니라 **동물의 시각 피질 실험**에서 출발

단순 세포와 복합 세포



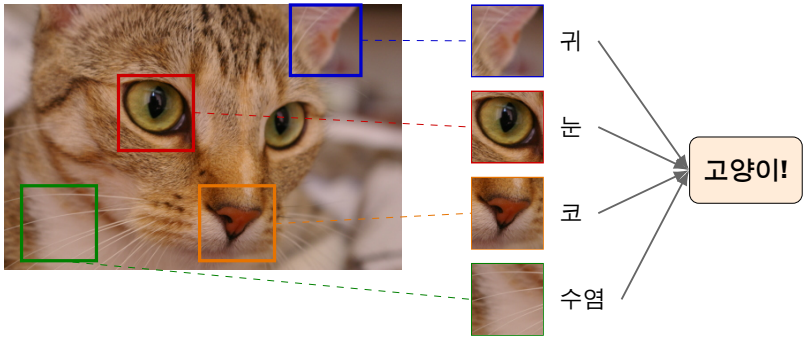
- 점선 원 = 뉴런의 **수용 영역(receptive field)**; 세로 선 하나 = spike 하나 (두 뉴런 모두 세로 방향 선호)
- **단순 세포**: 특정 **위치**의 특정 **방향** 에지에만 반응 ⇒ Sobel 같은 **에지 필터**
- **복합 세포**: 방향만 맞으면 위치가 조금 달라도 반응 ⇒ **최대 풀링**
- 두 세포는 **단계로 이어진다**: 점 신호 → (모아서) 단순 세포 → (모아서) 복합 세포 = **계층 모델**
- 이 쌓기가 V1 → V2 → V4 → 하측두엽으로 계속: 보는 범위가 넓어지고 모서리, 모양, 얼굴·사물에 반응 (1981년 노벨 생리의학상)

시각 피질에서 CNN으로: CNN의 계보



- Fukushima **Neocognitron** (1980): S-층(단순 세포)과 C-층(복합 세포)을 교대로; 가중치는 자기조직화(비지도)
- LeCun (1989): **역전파** + **가중치 공유** → **LeNet** (1998) → GPU·ImageNet과 만나 **AlexNet** (2012)

사람은 부분을 보고 전체를 판단한다

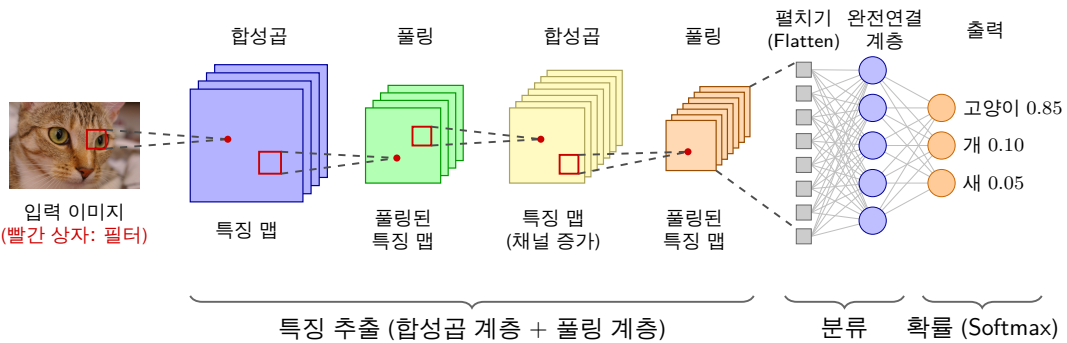


13만 개의 픽셀을 한꺼번에 비교하지 않는다: 부분(귀, 눈, 코, 수염)을 먼저 알아보고, 모아서 “고양이”로 판단. 각 부분은 **주변의 좁은 영역**만 보면 된다.

CNN은 이 과정을 모델링한다

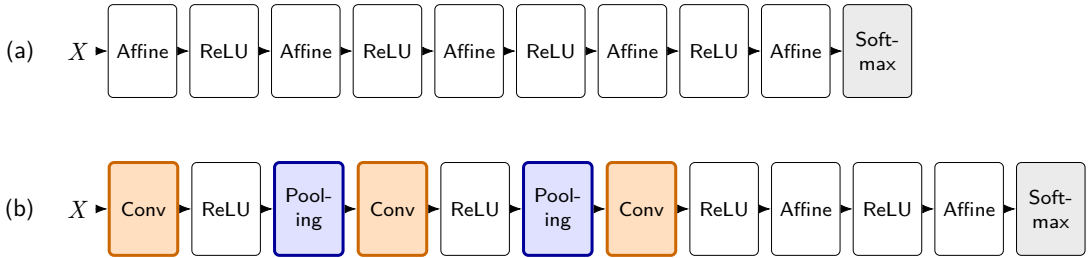
- **좁은 영역씩 본다**: 합성곱 계층의 출력 하나는 입력의 작은 영역 (3×3, 5×5)에만 연결
- **같은 검출기를 어디에나 쓴다**: “눈 검출기”는 왼쪽 눈에도 오른쪽 눈에도 $\Rightarrow$ 필터 하나를 모든 위치로 밀며 적용
- **단순한 것에서 복잡한 것으로**: 앞쪽 계층은 에지·색, 뒤쪽 계층은 눈·귀 같은 부분, 마지막 완전연결 계층이 “고양이”를 판단

전형적인 CNN의 구조



- 입력의 작은 영역 × 필터(빨간 상자, 가중치)의 합 → 다음 특징 맵의 한 점; **필터 하나 = 특징 하나를 뽑는 가중치**
- 합성곱·풀링을 거듭할수록 특징 맵은 **작아지고**, 개수(채널)는 **많아진다**
- 마지막 특징 맵을 펼친 뒤(Flatten) 지금까지 배운 완전연결 계층과 Softmax로 분류

완전연결 신경망과 CNN의 계층 구성



- (a) 완전연결 신경망(FNN): 인접 계층의 모든 노드가 연결 → **Affine** 계층으로 구현
- (b) CNN: "Affine → ReLU" 대신 "**Conv** → ReLU → (**Pooling**)" (풀링은 생략하기도 함)
- 출력 가까이는 "Affine → ReLU", 마지막은 "Affine → Softmax" ⇒ 앞부분은 특징 추출, 뒷부분은 분류

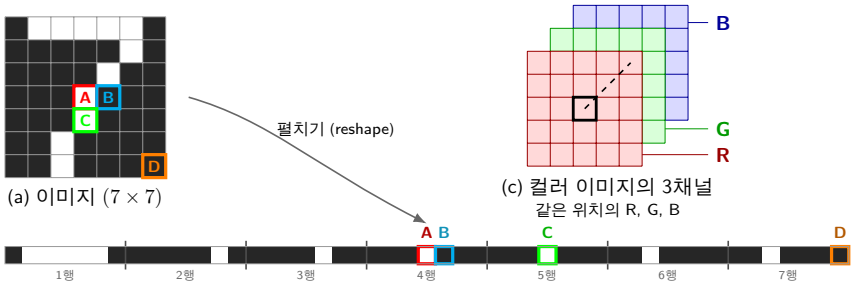
2

합성곱 계층

완전연결 계층의 문제점

- **데이터의 형상이 무시된다**
  - MNIST 이미지는 (채널, 세로, 가로) $= (1, 28, 28)$인 3차원 데이터
  - Affine 계층에 넣으려면 784개의 값을 한 줄로 펼쳐야 했다
- 이미지의 **3차원 모양에 담긴 패턴**
  - 공간적으로 가까운 픽셀끼리 값이 비슷하고, 떨어진 픽셀끼리는 관련이 적다
  - RGB의 세 채널은 서로 밀접하게 관련
- 완전연결 계층은 모든 입력을 같은 차원의 값으로 취급
  - 값은 그대로 남지만 (reshape로 복원 가능), 이웃 관계에 대한 가정이 없다
  - 모든 이미지의 픽셀을 같은 순서로 섞어도 표현할 수 있는 함수는 같다
  - 이웃 관계를 처음부터 데이터로 배워야 한다

펼치면 드러나지 않는 이웃 관계



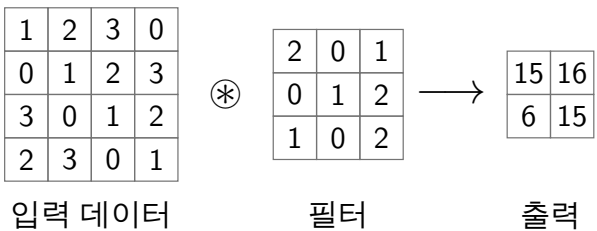
(a) 이미지 (7×7)

(c) 컬러 이미지의 3채널
같은 위치의 R, G, B

(b) 한 줄로 펼친 벡터 (49)

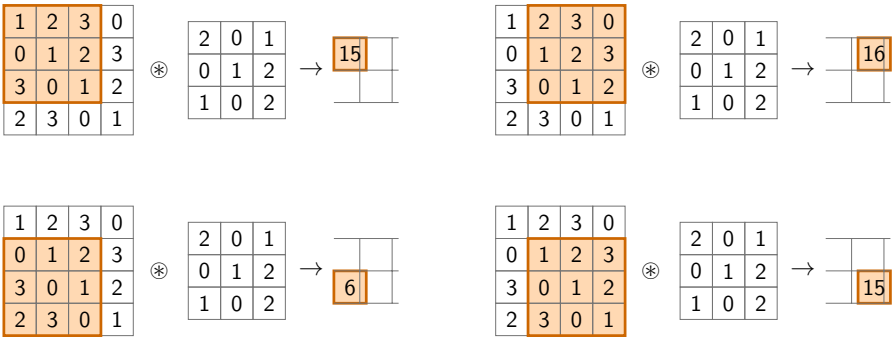
- A의 오른쪽 이웃 B는 옆에 남지만, 아래 이웃 C는 7칸 (MNIST라면 28칸) 떨어진다; (c) 같은 위치의 R, G, B는 한 점의 색
- 합성곱 계층은 **형상을 유지**: 입출력 데이터 = **특징 맵(feature map)** (입력 특징 맵, 출력 특징 맵)

합성곱 연산: 채널이 하나인 경우



- 입력 $(4, 4)$, 필터 $(3, 3)$ $\rightarrow$ 출력 $(2, 2)$
- 필터 $=$ **커널(kernel)**
- 필터의 **윈도우**를 일정 간격으로 이동
- 각 위치에서 **대응 원소끼리 곱한 뒤 모두 더함** (곱셈-누산)
- 필터를 뒤집지 않음: 엄밀히는 상호상관(cross-correlation)이지만 관례상 합성곱
- 왼쪽 위 $\rightarrow$ 오른쪽 아래, 모든 위치에서 반복

합성곱 연산의 계산 순서



- 첫 위치: $1 \cdot 2 + 2 \cdot 0 + 3 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 2 \cdot 2 + 3 \cdot 1 + 0 \cdot 0 + 1 \cdot 2 = 15$
- 입력 x, 필터 $f\,(FH \times FW)$에 대하여 $\quad y_{ij} = \sum_{p=0}^{FH-1} \sum_{q=0}^{FW-1} x_{i+p,\, j+q} f_{pq}$

사람이 설계한 필터: 이미지 처리에서의 특징 추출

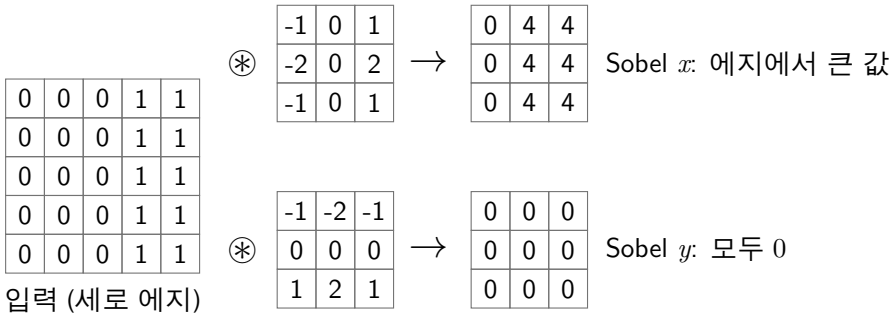
- 방금 본 계산 (작은 행렬 $=$ **필터**를 밀며 곱해 더하기)은 CNN 이전부터 이미지 처리에서 써 왔다
- 어떤 특징을 뽑을지는 필터의 값이 정하고, 그 값은 **사람이 설계**

필터	값	하는 일
평균 (blur)	$\frac{1}{25}\begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}_{5\times 5}$	주변 25개 픽셀의 평균. 잡음과 세부가 사라지고 흐려진다.
선명하게 (sharpen)	$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$	가운데 픽셀과 이웃의 차이를 키운다. 윤곽이 또렷해진다.
Sobel x	$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$	오른쪽 열 $-$ 왼쪽 열: 가로 방향 밝기 변화, 즉 **세로 에지**에 반응한다.

사람이 설계한 필터 (계속)

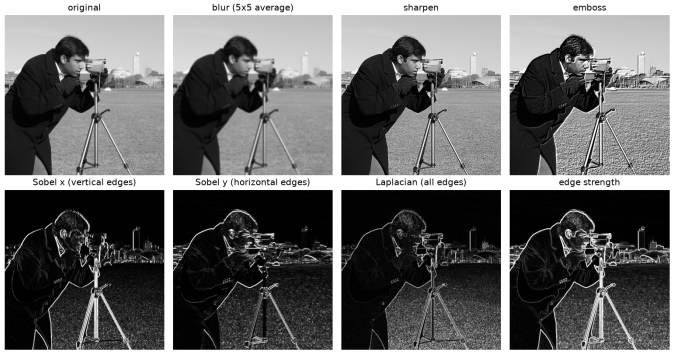
필터	값	하는 일
Sobel y	$\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$	아래 행 − 위 행: 세로 방향 밝기 변화, 즉 **가로 에지**에 반응한다.
Laplacian	$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$	이웃의 합 − $4\times$ 가운데 (2계 미분의 근사): 방향에 관계없이 에지에 반응한다.
엠보싱 (emboss)	$\begin{pmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix}$	대각선 방향의 밝기 변화를 더해 돋을새김처럼 보이게 한다.

왜 이런 필터가 에지에 반응할까?



- 왼쪽 절반 0(검정), 오른쪽 절반 1(흰색)인 5×5 이미지; Sobel 원소의 합 $= 0 \Rightarrow$ 밝기가 고른 곳은 출력 0
- Sobel x: $0/1$ 경계가 창 안에 있을 때만 $(0 \cdot (-1) + 1 \cdot 1)(1 + 2 + 1) = 4$; Sobel y: 위아래 변화가 없어 모두 0
- 출력 $=$ 그 위치의 이미지 조각이 **필터의 무늬와 얼마나 닮았는가**

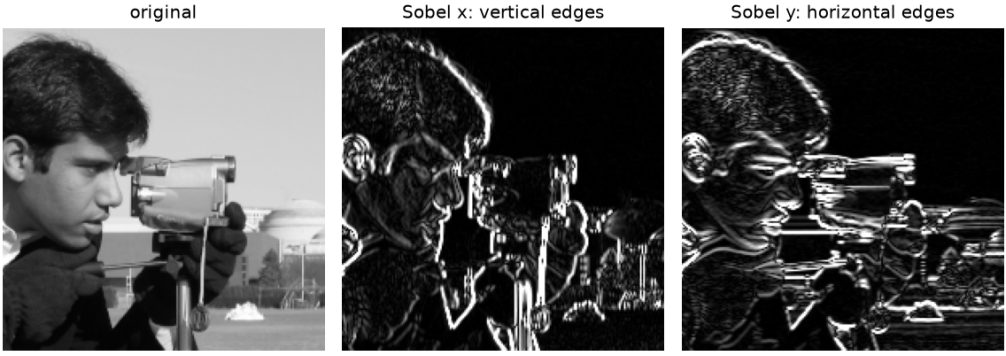
사람이 설계한 필터를 사진에 적용하면



`classic_filters.py` (512×512 흑백 사진). 위: 원본, 평균, 선명하게, 엠보싱. 아래: Sobel x, Sobel y, Laplacian, 에지의 세기 $\sqrt{g_x^2 + g_y^2}$
(에지 필터의 출력은 양수·음수가 모두 나오므로 절댓값; 밝을수록 강한 에지)

- g_x, g_y = 같은 위치에서 Sobel x, Sobel y의 출력 (가로·세로 방향의 밝기 변화량) → 에지의 세기 $\sqrt{g_x^2 + g_y^2}$ = 벡터 (g_x, g_y)의 길이: 어느 방향의 에지든 밝게

Sobel x와 Sobel y의 차이 (확대)



- Sobel x: 삼각대 다리, 얼굴의 옆선처럼 **세로로 뻗은 경계**가 밝다
- Sobel y: 카메라의 윗면·아랫면, 지평선처럼 **가로로 뻗은 경계**가 밝다
- 필터 하나가 이미지에서 **한 종류의 특징**만 골라낸다

사람이 설계한 필터는 지금 어디에 쓰이는가

사람이 설계한 필터는 지금 어디에 쓰이는가

딥러닝이 대신한 것은 "이것이 고양이인가" 같은 복잡한 인식. 단순하고 목적이 분명한 처리에는 지금도 고정된 필터를 쓴다.

- **사진·영상 편집과 카메라**: 흐림 효과, 선명하게, 잡음 제거, 윤곽 보정, 축소 전 블러(앨리어싱 방지)
- **고전 컴퓨터 비전**: Canny (가우시안 블러 $\rightarrow$ Sobel), Harris 코너, HOG·SIFT (HOG + SVM 보행자 검출)
- **산업 현장**: 표면 긁힘 검사, 문서 스캔·OCR 전처리, 차선 인식 (불량 데이터가 적거나 GPU 없는 임베디드)
- **딥러닝 안에서**: 데이터 증강, 흐린 사진 걸러내기 (Laplacian 분산), Sobel 에지 손실, BlurPool

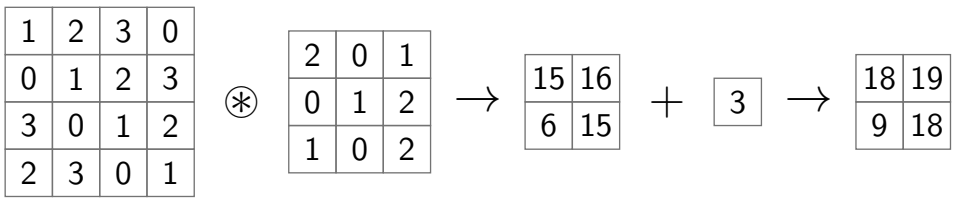
고정된 필터는 **데이터가 필요 없고, 해석할 수 있고, 빠르다** — 대신 사람이 수식으로 정의할 수 있는 특징까지만.

사람이 설계한 필터에서 학습하는 필터로

사람이 설계한 필터에서 학습하는 필터로

- 필터 연산 = **작은 무늬 검출기**; 출력은 각 위치에서 그 무늬가 얼마나 강한지 담은 2차원 지도, 즉 **특징 맵**
- 복잡한 이미지를 분류하려면 어떤 필터를, 어떤 값으로 써야 할지 사람이 정하기 어렵다
- CNN은 필터의 값을 **데이터로부터 학습**하는 **가중치**로 취급 (필터 = 학습 파라미터)
- 학습된 첫 층의 필터도 Sobel처럼 여러 방향의 에지에 반응하고, 층이 깊어질수록 더 복잡한 무늬를 찾는다

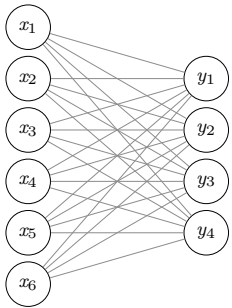
필터가 가중치이다



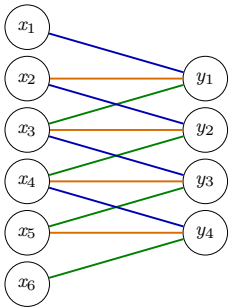
- 완전연결 신경망: 가중치 W와 편향 B
- CNN: **필터의 원소가 가중치** $\Rightarrow$ 필터의 값을 학습한다
- 편향: 필터를 적용한 뒤 **모든 원소에 같은 값 하나**를 더한다

합성곱 계층은 연결을 제한한 완전연결 계층이다

1차원 입력 $x_1, \ldots, x_6$, 길이 3인 필터 (w_1, w_2, w_3), 스트라이드 1:
$y_j = w_1 x_j + w_2 x_{j+1} + w_3 x_{j+2} \quad (j = 1, \ldots, 4)$



(a) 완전연결
가중치 $6 \times 4 = 24$개



(b) 합성곱
가중치 3개 (색이 같으면 같은 값)

$$W = \begin{bmatrix} w_1 & & & \\ w_2 & w_1 & & \\ w_3 & w_2 & w_1 & \\ & w_3 & w_2 & w_1 \\ & & w_3 & w_2 \\ & & & w_3 \end{bmatrix}$$

(c) (b)의 가중치 행렬 (6×4)
빈칸은 0

$Y = XW$로 쓰면 합성곱은 W의 대부분이 0이고 같은 값이 대각선을 따라 반복되는 특별한 완전연결 계층

완전연결 계층 + 두 가지 규칙 = 합성곱 계층

- **국소 연결 (sparse connectivity)**
  - y_j는 **공간적으로 인접한** x_j, x_{j+1}, x_{j+2}에만 연결 $\rightarrow$ 이 영역이 y_j의 **수용 영역**
  - 이미지의 기하 구조(가까운 픽셀끼리 관련이 깊다)를 연결 방식에 반영
- **가중치 공유 (parameter sharing)**
  - 모든 출력 노드가 **같은 가중치** (w_1, w_2, w_3) $\rightarrow$ 무늬가 어디에 있든 같은 방식으로 찾는다
- W: 각 열에 w_1, w_2, w_3이 한 칸씩 내려가며 놓이는 **띠 모양 행렬**
- 2차원 예: MNIST (28×28), 5×5 필터 하나 $\rightarrow$ 24×24
  - 완전연결: $784 \times 576 = 451584$개 vs. 합성곱: **25**개

참고: Goodfellow, Bengio, Courville, *Deep Learning* 9장 (sparse interactions, parameter sharing)

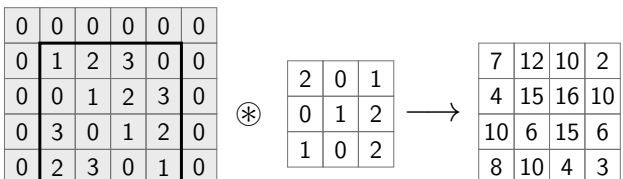
코드로 확인하기: conv2d_naive (conv_basics.py)

```python
def conv2d_naive(x, f, stride=1, pad=0):
    """2-D convolution of one image x (H, W) with one filter f (FH, FW), with loops"""
    x = np.pad(x, pad)                                  # zeros around the image
    FH, FW = f.shape
    OH = (x.shape[0] - FH) // stride + 1
    OW = (x.shape[1] - FW) // stride + 1
    y = np.zeros((OH, OW))
    for i in range(OH):
        for j in range(OW):
            window = x[i * stride:i * stride + FH, j * stride:j * stride + FW]
            y[i, j] = np.sum(window * f)                # multiply elementwise, then add
    return y
```

```
convolution (4x4 input, 3x3 filter):
 [[15. 16.]
 [ 6. 15.]]
```

```
with bias 3:
 [[18. 19.]
 [ 9. 18.]]
```

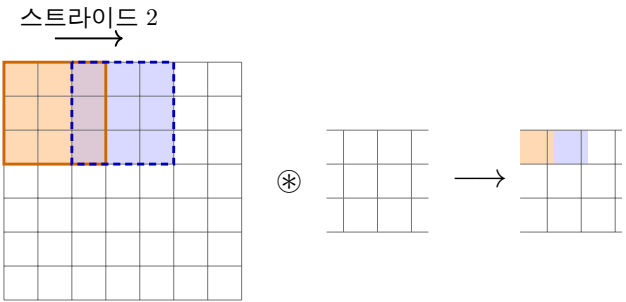
패딩 (padding)



$(4,4)$ 입력 + 패딩 1 $(3,3)$ 필터 $(4,4)$ 출력

- 합성곱 전에 입력 주변을 특정 값 (보통 0)으로 채움
- 패딩 없음: $(4,4) \to (2,2)$
  패딩 1: $(4,4) \to (4,4)$
- 주로 **출력 크기 조정**: 깊은 network에서 크기가 계속 줄어 1이 되는 것을 막음
- 가장자리 픽셀의 정보도 더 반영

스트라이드 (stride)



- 필터를 적용하는 위치의 간격
- 지금까지는 스트라이드 1;
  스트라이드 2이면 윈도우가 두 칸씩 이동
- 7×7, 3×3 필터:
  스트라이드 $1 \rightarrow (5,5)$,
  스트라이드 $2 \rightarrow (3,3)$
- 스트라이드 $\uparrow \Rightarrow$ 출력 $\downarrow$,
  패딩 $\uparrow \Rightarrow$ 출력 $\uparrow$

합성곱의 출력 크기

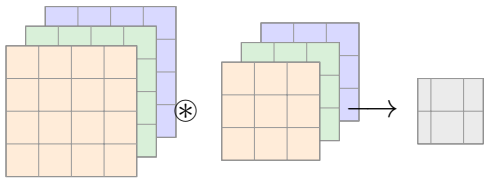
입력 (H, W), 필터 (FH, FW), 패딩 P, 스트라이드 S일 때 출력 (OH, OW):

합성곱의 출력 크기

$$OH = \left\lfloor \frac{H + 2P - FH}{S} \right\rfloor + 1, \qquad OW = \left\lfloor \frac{W + 2P - FW}{S} \right\rfloor + 1$$

- 입력 $(4, 4)$, $P = 1$, $S = 1$, 필터 $(3, 3)$: $OH = \frac{4+2-3}{1} + 1 = 4$
- 입력 $(7, 7)$, $P = 0$, $S = 2$, 필터 $(3, 3)$: $OH = \frac{7+0-3}{2} + 1 = 3$
- 입력 $(28, 31)$, $P = 2$, $S = 3$, 필터 $(5, 5)$: $OH = \frac{28+4-5}{3} + 1 = 10$, $OW = \frac{31+4-5}{3} + 1 = 11$
- $\lfloor \cdot \rfloor$ = 내림: 남는 부분에는 필터를 놓지 않는다 (예: $H = 8$, $FH = 3$, $S = 2 \rightarrow \lfloor 5/2 \rfloor + 1 = 3$). 코드의 `//`, PyTorch도 같다

3차원 데이터의 합성곱



입력 데이터 (채널 3개, 4×4) 필터 (채널 3개, 3×3) 출력 (2×2, 채널 1개)

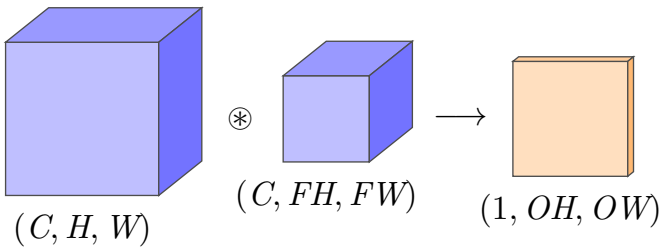
- 이미지는 세로·가로에 **채널**까지 3차원 (컬러는 RGB 3채널)
- **채널마다** 입력과 필터의 합성곱 → 결과를 **모두 더해** 하나의 출력

3차원 합성곱의 규칙

- 입력의 채널 수 = 필터의 채널 수
- 필터 크기 (FH, FW)는 자유, 단 모든 채널이 같은 크기
- 채널 C개 입력 ⊛ 필터 하나 → **채널 1개**

$$y_{ij} = \sum_{c=1}^{C} \sum_{p} \sum_{q} x_{c,\, i+p,\, j+q} f_{c,\, p,\, q} + b$$

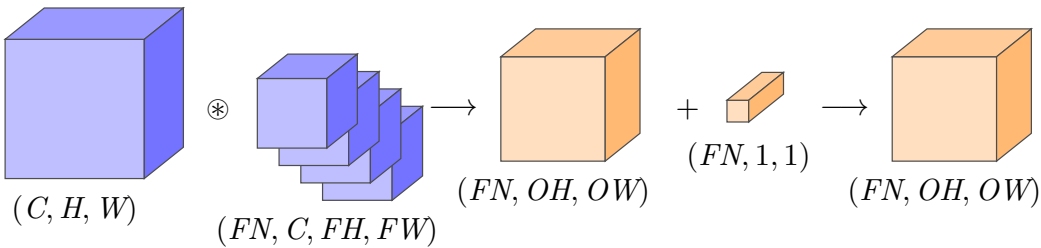
블록으로 생각하기



필터 1개 $\Rightarrow$ 특징 맵 1장

- 데이터와 필터를 직육면체 **블록**으로 생각한다
- 3차원 데이터의 shape: (채널, 높이, 너비) $= (C, H, W)$, 필터: (C, FH, FW)
- 필터 1개 $\rightarrow$ 출력 $(1, OH, OW)$: 다음 계층에 넘길 정보가 너무 적다

필터를 FN개 쓰는 합성곱 계층

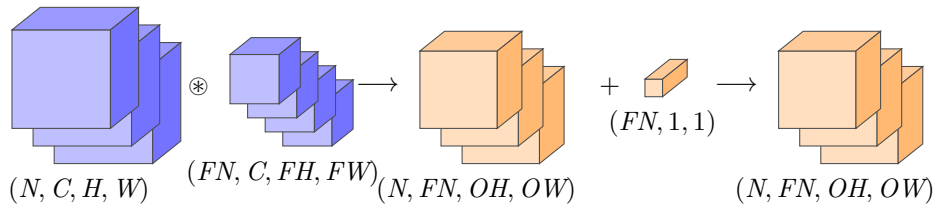


- 필터 FN개 $\rightarrow$ 출력 특징 맵 FN장: (FN, OH, OW) 블록
- 합성곱 계층의 필터(가중치)는 **4차원** (FN, C, FH, FW)
- 편향은 필터(출력 채널)마다 하나씩 FN개: shape $(FN, 1, 1)$로 보면 broadcast로 더해진다

왜 합성곱인가: 가중치 공유

- **같은 필터를 이미지의 모든 위치에 적용** (weight sharing)
  - 세로 획 필터 하나가 왼쪽 위의 세로 획도, 오른쪽 아래의 세로 획도 똑같이 찾는다
- 매개변수가 아주 적다: MNIST $(1, 28, 28) \to$ 출력 $(30, 24, 24)$
  - 완전연결 계층: $784 \times (30 \cdot 24 \cdot 24) = 784 \times 17280 \approx 1350$만 개의 가중치
  - 합성곱 계층 (5×5 필터 30개): $30 \times 1 \times 5 \times 5 = \mathbf{750}$개 가중치 $+ 30$개 편향
- 각 출력 원소는 입력의 5×5 영역만 본다 (**국소 연결**)
- 이 두 성질이 이미지의 구조에 잘 맞는다

배치 처리

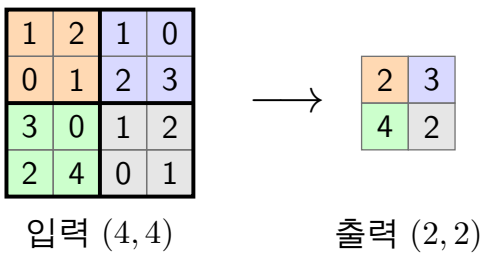


- 완전연결 계층의 미니배치처럼, 합성곱 계층도 **배치 처리**
- 데이터를 **4차원** (데이터 수, 채널, 높이, 너비) $= (N, C, H, W)$로 저장
  - 미니배치 크기를 N으로 표기 (높이 H와 혼동 방지)
- 4차원 데이터 하나가 흐를 때마다 N개에 대한 합성곱이 한 번에 이루어진다

3

풀링 계층

최대 풀링 (max pooling)



- **풀링**: 세로·가로 방향의 공간을 줄이는 연산
- 2×2 최대 풀링, 스트라이드 2: 2×2 영역마다 **가장 큰 원소** 하나
- 윈도우 크기와 스트라이드는 보통 같게 (윈도우 3×3이면 스트라이드 3)
- 영역의 평균을 내는 **평균 풀링**도 있지만, 이미지 인식에서는 주로 최대 풀링
- `pool = Pooling(2, 2)` → `[[2. 3.] [4. 2.]]`

풀링 계층의 특징

풀링 계층의 특징

- **학습해야 할 매개변수가 없다**
  - 영역에서 최댓값(또는 평균)을 취하는 명확한 처리일 뿐
- **채널 수가 변하지 않는다**
  - 채널마다 독립적으로 계산: $(N, C, H, W) \to (N, C, OH, OW)$
- **입력의 작은 변화에 영향을 적게 받는다 (robust)**
  - 입력이 한두 칸 이동해도 영역 안의 최댓값은 그대로인 경우가 많다
  - 시각 피질의 **복합 세포**와 같은 역할

4

합성곱 / 풀링 계층 구현하기

구현 파일

- 합성곱·풀링 계층도 `forward`와 `backward`를 가진 **클래스**로 구현
- Affine, ReLU, Softmax 계층은 지난 시간의 `layers.py`를 그대로 사용

파일	내용
`conv_layers.py`	`im2col`, `col2im`, `Convolution`, `Pooling`, `Flatten`, `SimpleConvNet`
`layers.py`, `optimizers.py`, `mnist_data.py`	지난 시간의 모듈 (그대로 복사)
`cat_parts.py`	도입부의 고양이 사진과 부분 그림
`classic_filters.py`	사람이 설계한 필터 (평균, Sobel, Laplacian 등)를 흑백 사진에 적용
`conv_basics.py`	이 노트의 예제, im2col 확인과 속도 비교, gradient 확인
`mnist_cnn.py`	SimpleConvNet으로 MNIST 학습, 필터와 특징 맵 시각화
`mnist_cnn_torch.py`	같은 network를 PyTorch로

4차원 배열

- 계층 사이를 흐르는 데이터: 4차원 배열 (N, C, H, W)
- 높이 28, 너비 28, 채널 1인 데이터 10개 $\rightarrow$ shape $(10, 1, 28, 28)$

```
>>> x = np.random.rand(10, 1, 28, 28)
>>> x[0].shape         # the first image: (1, 28, 28)
>>> x[0, 0].shape      # its first channel: (28, 28)
```

- 지금까지 MNIST는 $(N, 784)$ $\rightarrow$ CNN에서는 `reshape`으로 모양을 되살려 입력 (`mnist_cnn.py`)

```
x_train = x_train.reshape(-1, 1, 28, 28)   # (N, 784) -> (N, 1, 28, 28)
```

im2col로 데이터 전개하기

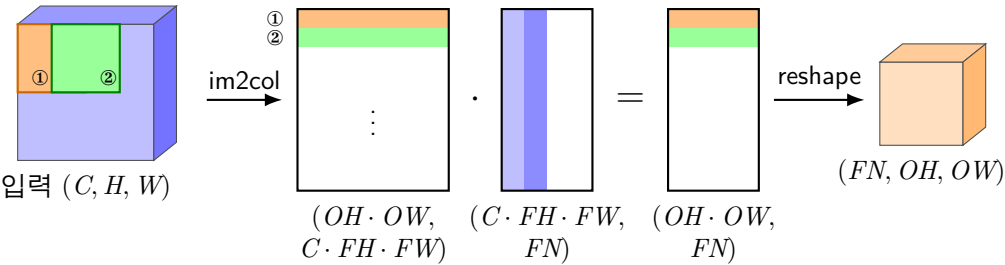
- `conv2d_naive`처럼 반복문으로 구현하면: 데이터 N개 $\times$ 필터 FN개 $\times$ 출력 위치 $OH \times OW \rightarrow$ `for` 문이 겹겹이 (numpy에서 느림)
- **im2col** (image to column): 입력을 필터 적용에 좋게 **2차원 행렬로 펼친다**

im2col의 아이디어

필터가 방문하는 위치 하나마다, 그 위치의 입력 영역 (C, FH, FW)을 **한 행**으로 펼친다.

- 입력 $(N, C, H, W) \xrightarrow{\text{im2col}}$ 행렬 $(N \cdot OH \cdot OW,\ C \cdot FH \cdot FW)$
- 필터 $(FN, C, FH, FW) \xrightarrow{\text{reshape}}$ 필터 하나를 **한 열**로: $(C \cdot FH \cdot FW,\ FN)$
- 합성곱 $=$ 두 행렬의 곱 $+$ 편향: $(N \cdot OH \cdot OW,\ FN) \xrightarrow{\text{reshape}} (N, FN, OH, OW)$

im2col을 이용한 합성곱



데이터 하나의 경우. 필터의 위치 ①, ②, . . . 이 행렬의 각 행이 된다.

- 합성곱 계층의 순전파 = **Affine 계층과 똑같은** 행렬곱 $A = \text{col} \cdot W_{\text{col}} + B$
- 행렬곱은 최적화된 선형대수 라이브러리(BLAS)로 계산 → 매우 빠르다

im2col 구현

```
def im2col(X, FH, FW, stride=1, pad=0):
    """(N, C, H, W) -> (N*OH*OW, C*FH*FW): ONE ROW for every place the filter visits"""
    N, C, H, W = X.shape
    OH, OW = output_size(H, FH, stride, pad), output_size(W, FW, stride, pad)
    Xp = np.pad(X, [(0, 0), (0, 0), (pad, pad), (pad, pad)])    # zero padding
    col = np.zeros((N, C, FH, FW, OH, OW))
    for i in range(FH):                    # loop over the FILTER, not the image
        for j in range(FW):
            col[:, :, i, j, :, :] = Xp[:, :, i:i + stride * OH:stride, j:j + stride * OW:stride]
    return col.transpose(0, 4, 5, 1, 2, 3).reshape(N * OH * OW, -1)
```

- 반복은 이미지 전체가 아니라 **필터의 크기** $FH \times FW$ (예: 25번)만큼만
- `(i, j)`번째 반복: 필터의 (i, j) 원소와 곱해질 입력 값들을 모든 출력 위치에 대해 한꺼번에 잘라 옴
- $(1, 3, 7, 7)$, 5×5 필터 $\rightarrow (9, 75)$: 출력 위치 $3 \times 3 = 9$, 영역 $3 \cdot 5 \cdot 5 = 75$; 데이터 10개 $\rightarrow (90, 75)$

얼마나 빠를까? 그리고 그 대가

- $(100, 1, 28, 28)$ 미니배치, 5×5 필터 30개: 네 겹 반복문 (`conv_loops`) vs. im2col
  - 출력 $(100, 30, 24, 24)$, 결과 차이 5.3×10^{-15}
  - 반복문 5.74초, im2col 0.057초 $\Rightarrow$ **약** 100**배** 빠름
  - 반복문이면 550개 미니배치의 한 epoch에 거의 1시간

im2col의 대가: 메모리

- 필터가 겹쳐 이동하면 입력의 같은 값이 여러 행에 **중복**해서 들어간다
- $(100, 1, 28, 28)$ (78400개) $\rightarrow (100 \cdot 24 \cdot 24,\ 25) = (57600, 25)$, 144만 개: 약 18배
- 메모리를 더 쓰는 대신 속도를 얻는다 (`accuracy`는 1000장씩 나누어 계산)
- GPU 라이브러리(cuDNN 등)는 im2col 외에 FFT, Winograd 등을 골라 쓴다

합성곱 계층 구현: Convolution.forward

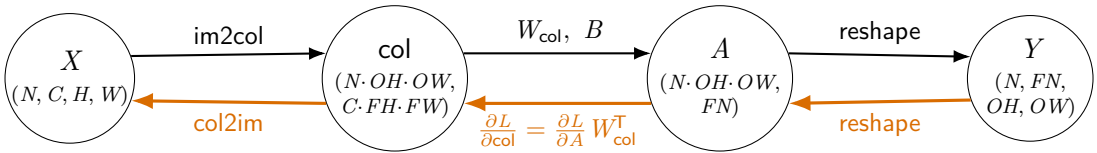
im2col을 쓰면 순전파는 “im2col → Affine → reshape”

```
class Convolution:    # self.W: (FN, C, FH, FW), self.B: (FN,) one bias per filter
    def forward(self, X):
        FN, C, FH, FW = self.W.shape
        N, _, H, W = X.shape
        OH = output_size(H, FH, self.stride, self.pad)
        OW = output_size(W, FW, self.stride, self.pad)
        self.X_shape = X.shape
        self.col = im2col(X, FH, FW, self.stride, self.pad)      # (N*OH*OW, C*FH*FW)
        self.W_col = self.W.reshape(FN, -1).T                     # one filter per COLUMN
        A = self.col @ self.W_col + self.B                        # (N*OH*OW, FN)
        return A.reshape(N, OH, OW, FN).transpose(0, 3, 1, 2)     # (N, FN, OH, OW)
```

- `A`의 각 행 = (데이터, 출력 위치) 하나, 각 열 = 필터(출력 채널) 하나 → 채널 축을 두 번째로
- `output_size`: `(H + 2*pad - FH) // stride + 1` (출력 크기 공식 그대로)

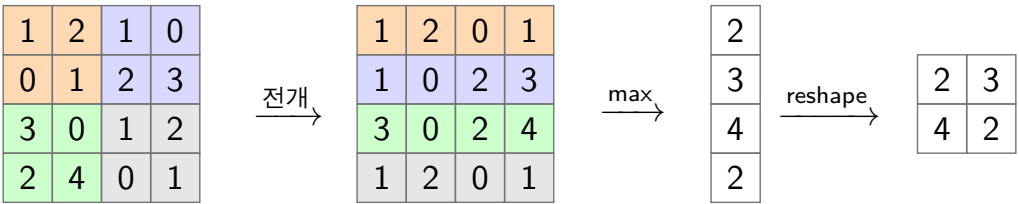
합성곱 계층의 역전파

$$\frac{\partial L}{\partial W_{\text{col}}} = \text{col}^{\mathsf{T}} \frac{\partial L}{\partial A}, \quad \frac{\partial L}{\partial B} = \texttt{np.sum}\left(\frac{\partial L}{\partial A}, \texttt{axis=0}\right)$$



- **reshape**의 역전파: gradient의 모양만 원래대로 (값은 그대로)
- **가운데는 Affine 계층**: $\frac{\partial L}{\partial W} = X^{\mathsf{T}} \frac{\partial L}{\partial A}$, $\frac{\partial L}{\partial B} = \sum_{\text{행}} \frac{\partial L}{\partial A}$, $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial A} W^{\mathsf{T}}$ 그대로
- **im2col**의 역전파 = **col2im**: 입력의 한 값이 col의 여러 행에 복사되었으므로 gradient를 **모두 더한다**
  (im2col의 = 대신 col2im은 +=; 다변수 연쇄법칙 — 배치 정규화와 같은 원리)

풀링 계층 구현하기



- 풀링도 im2col로 입력을 전개
- 풀링은 **채널마다 독립적** → 한 행이 "한 채널의 한 윈도우"가 되도록 다시 `reshape`
- 행마다 최댓값을 구한 뒤 모양을 되돌린다

Pooling.forward와 풀링의 역전파

```
    def forward(self, X):
        N, C, H, W = X.shape
        OH = output_size(H, self.PH, self.stride, self.pad)
        OW = output_size(W, self.PW, self.stride, self.pad)
        col = im2col(X, self.PH, self.PW, self.stride, self.pad)
        col = col.reshape(-1, self.PH * self.PW)            # one window per row
        self.argmax = np.argmax(col, axis=1)                # where the maximum was
        self.X_shape = X.shape
        Y = np.max(col, axis=1)
        return Y.reshape(N, OH, OW, C).transpose(0, 3, 1, 2)    # (N, C, OH, OW)
```

- $y = \max(x_1, x_2, x_3, x_4)$에서 최댓값이 x_k이면 $\frac{\partial y}{\partial x_k} = 1$, 나머지는 0
- **최댓값이었던 원소에만** gradient를 그대로 전달, 나머지에는 0 (ReLU의 `mask`와 같다)
- 그래서 순전파에서 최댓값의 위치 `argmax`를 기억해 둔다 (backward: `dcol[arange, argmax] = dY` → col2im)
- 최댓값이 여러 개로 같으면: `np.argmax`가 고른 첫 위치로 보낸다

Flatten 계층

- 합성곱·풀링을 거친 4차원 데이터를 Affine 계층에 넣으려면 데이터마다 한 줄로 펼쳐야 한다
- 이것을 계층 하나로 (PyTorch의 `nn.Flatten`); 역전파는 모양을 되돌리는 것뿐

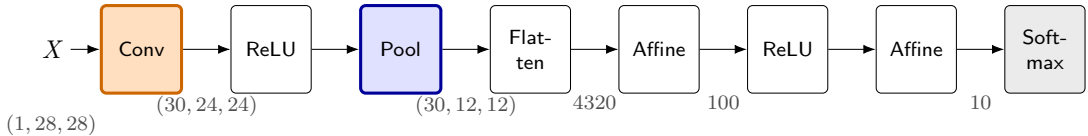
```
class Flatten:
    """(N, C, H, W) -> (N, C*H*W) before an Affine layer (nn.Flatten in PyTorch)"""
    def forward(self, X):
        self.X_shape = X.shape
        return X.reshape(X.shape[0], -1)

    def backward(self, dY):
        return dY.reshape(self.X_shape)
```

5

CNN 구현하기

SimpleConvNet의 구조



SimpleConvNet의 구조와 각 계층을 지난 뒤 데이터 하나의 shape

- 지난 시간의 `MultiLayerNet`과 같은 방식
- 매개변수는 `params` 딕셔너리, 같은 배열을 계층에도 넘김 → optimizer가 갱신하면 계층 가중치도 바뀜
- 초깃값 `weight_init_std`: `"he"`이면 He ($C \cdot FH \cdot FW = 25 \rightarrow \sqrt{2/25}$), 숫자이면 그 표준편차
- 계층이 모두 같은 `forward`/`backward` 형식 → 새 계층이 들어와도 network 코드는 그대로

SimpleConvNet의 계층별 shape과 매개변수 수

계층	설정	출력 shape	매개변수 수
입력		$(1, 28, 28)$	
Conv	5×5 필터 30개	$(30, 24, 24)$	$30 \cdot 1 \cdot 5 \cdot 5 + 30 = 780$
ReLU		$(30, 24, 24)$	
Pooling	2×2 최대 풀링	$(30, 12, 12)$	
Flatten		4320	
Affine		100	$4320 \cdot 100 + 100 = 432100$
ReLU		100	
Affine		10	$100 \cdot 10 + 10 = 1010$
합계			433890

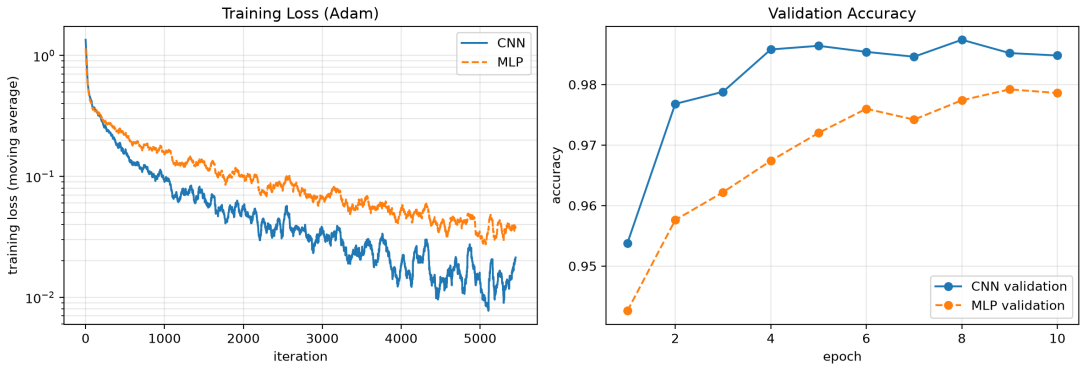
합성곱은 스트라이드 1, 패딩 0; 풀링은 스트라이드 2; 출력 shape은 데이터 하나의 것

- 합성곱 계층의 매개변수는 **780**개뿐, 매개변수의 99.6%는 첫 번째 Affine 계층에 (가중치 공유 덕분)

gradient 확인과 MNIST 학습 설정

- **gradient 확인** (`conv_basics.py`의 (3))
  - 작은 SimpleConvNet: 입력 $(2, 8, 8)$, 3×3 필터 3개, 데이터 5개
  - W1 $(3, 2, 3, 3)$, B1, W2 $(27, 5)$, B2, W3 $(5, 4)$, B3 모두 $\max |\text{backprop} - \text{numerical}| \approx 10^{-9}$
  - 합성곱, 풀링, Flatten을 지나는 역전파가 모두 올바르다
- **MNIST로 학습하기** (`mnist_cnn.py`)
  - SimpleConvNet (5×5 필터 30개, 은닉 100)을 Adam (학습률 0.001, 미니배치 100)으로 10 epoch
  - 초깃값은 표준편차 0.01로 작게 시작 $\rightarrow$ 학습으로 생긴 필터의 무늬가 잘 드러난다
  - 비교: 지난 시간의 MLP ($784 \rightarrow 100 \rightarrow 10$, He 초깃값)를 같은 조건으로

학습 결과: SimpleConvNet vs. MLP



훈련 손실 (50 iteration 이동평균, 로그 눈금, 왼쪽)과 epoch별 검증 정확도 (오른쪽)

학습 결과 비교

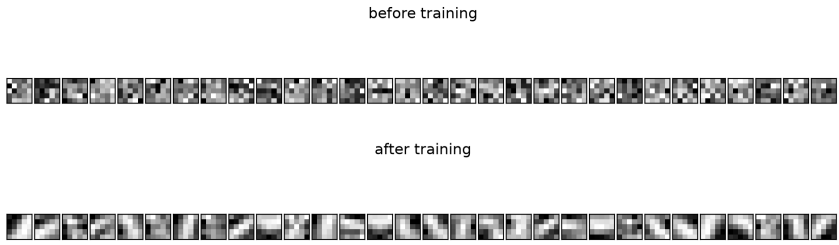
	검증 정확도						
	1 epoch	2 epoch	5 epoch	10 epoch	시험	매개변수	epoch당
SimpleConvNet	95.4%	97.7%	98.6%	98.5%	**98.5%**	433890	약 60초
MLP ($784 \to 100 \to 10$)	94.3%	95.8%	97.2%	97.9%	97.6%	79510	약 1초

- CNN은 2 **epoch 만에** 97.7% ≈ MLP의 10 epoch 97.9%; 시험 오류 237 → 154장 (약 35% 감소)
- 손실도 CNN이 훨씬 빨리, 훨씬 낮게 내려간다
- 매개변수 수가 달라 공정한 비교는 아님 — 합성곱 계층의 780개 매개변수가 **좋은 특징**을 만들어 뒤의 Affine 계층의 분류를 쉽게 한다
- numpy CNN은 epoch당 약 1분 → GPU와 최적화된 라이브러리가 중요 (20 epoch이면 시험 약 99%)

6

CNN 시각화하기

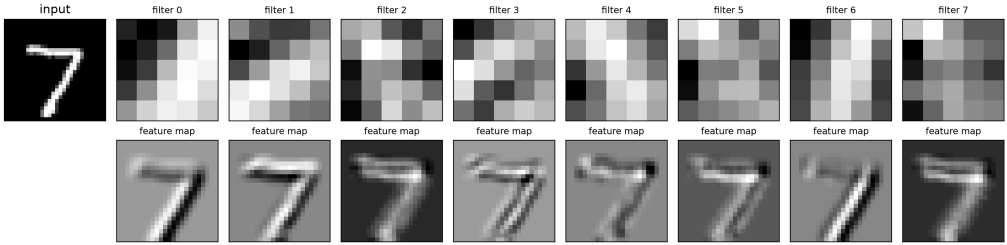
첫 번째 층의 가중치 시각화하기



첫 번째 합성곱 계층의 필터 30개 $(30, 1, 5, 5)$: 학습 전 (위)과 학습 후 (아래), 밝을수록 큰 가중치

- 학습 전: 무작위 $\rightarrow$ 학습 후: 흰색과 검은색이 **규칙적으로 나뉜** 모양
- 세로 경계, 가로 경계, 대각선 띠 $\rightarrow$ **에지**에 반응하는 필터
- Sobel처럼 사람이 설계했던 필터를 network가 데이터로부터 **스스로** 찾아냈다

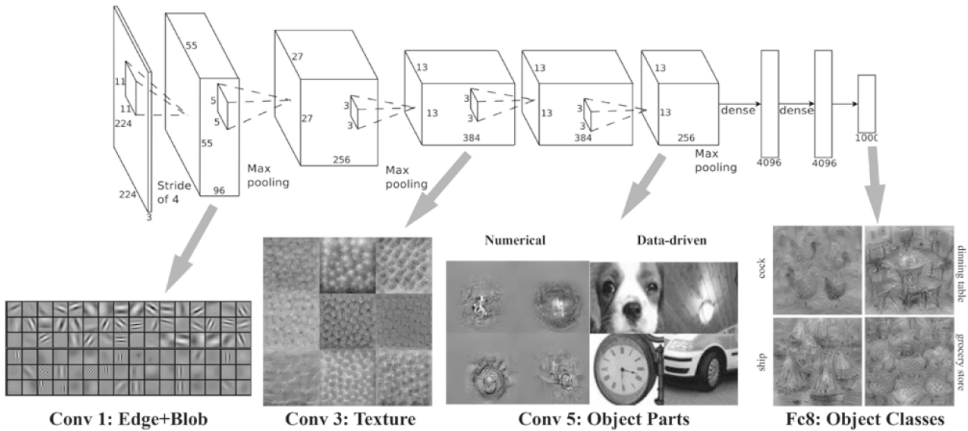
필터는 무엇에 반응하는가: 특징 맵



입력 이미지와 학습된 필터 8개 (위), 각 필터의 출력 특징 맵 (아래, ReLU 적용 전):
`cnn.layers[0].forward(image)` $\rightarrow (1, 30, 24, 24)$

- 특징 맵마다 숫자 7의 **서로 다른 부분이 밝게**: 필터 0은 대각선 획의 가장자리, 필터 1은 위쪽 가로 획의 아래 가장자리, 필터 6은 대각선 획 자체
- 첫 층의 필터는 특정 방향의 **에지**에 반응 — 모든 위치에 적용되므로 어디에 있든 찾는다
- 30장의 특징 맵을 풀링으로 줄인 뒤 Affine 계층이 조합하여 숫자를 판단

층 깊이에 따른 추출 정보 변화



ImageNet으로 학습한 8층 CNN (AlexNet)의 각 층 뉴런이 강하게 반응하는 이미지

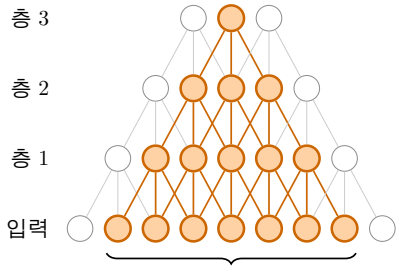
층이 깊어질수록: 단순한 모양 → 의미

- **Conv 1**: 에지와 블롭 (Sobel 같은 방향 필터)
- **Conv 3**: 텍스처 (벌집, 비늘 같은 반복 무늬)
- **Conv 5**: 사물의 일부 (개 얼굴, 시계, 바퀴)
  - Numerical: 뉴런 출력이 최대가 되도록 계산으로 만든 입력
  - Data-driven: 실제 데이터 중 가장 강하게 반응한 이미지
- **Fc8**: 사물의 클래스 (닭, 식탁, 배, 식료품점)
- 층이 깊어질수록 단순한 모양 → "고급" 정보, 즉 사물의 **의미**에 반응한다

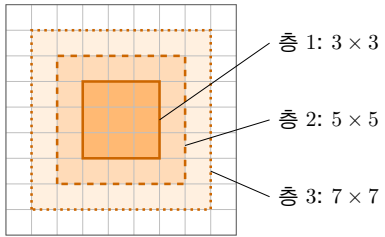
층을 깊게 쌓는 이유

- **단순한 특징을 조합하여 점점 추상적인 특징을 만드는 것** = 층을 깊게 쌓는 이유
- 합성곱을 거듭할수록 출력 한 원소의 **수용 영역**(receptive field)이 넓어진다 → 뒤쪽 층은 넓은 영역을 보고 판단
- 고양이 그림 (1절)의 "에지 → 부분 → 전체"를 실제 network가 데이터로부터 스스로 만든다

뒤쪽 층으로 갈수록 넓어지는 수용 영역



(a) 필터 길이 3인 합성곱을 세 번 (1차원)



(b) 같은 일을 이미지 (2차원)에서

- 층 3의 원소 하나 (맨 위 주황색)는 입력 7칸에 의존한다
- 3×3 필터 (스트라이드 1)를 L번 쌓으면 수용 영역은 $(2L+1) \times (2L+1)$
- 중간에 2×2 풀링 (스트라이드 2)이 있으면 훨씬 빨리 넓어진다 $\rightarrow$ 뒤쪽 층은 이미지의 큰 부분이나 전체를 본다

7

대표적인 CNN

CNN 이전의 문자 인식: 사람이 설계한 특징

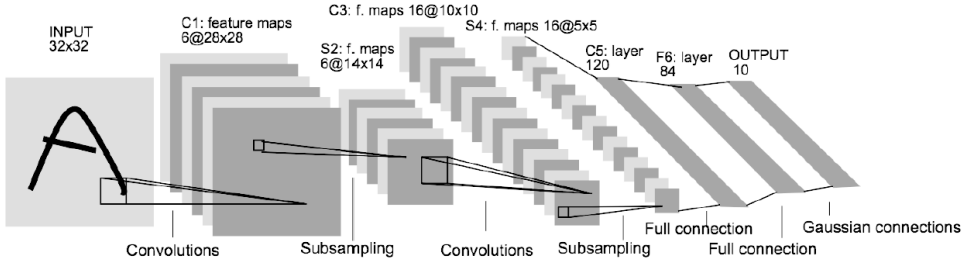
CNN 이전의 문자 인식: 사람이 설계한 특징

1990년대까지의 주류: **"사람이 설계한 특징 추출기 + 학습하는 분류기"**

- **템플릿 매칭 (1950–60년대)**: 글자마다 표준 모양과 비교 (템플릿 = 큰 필터) — 위치·크기·글씨체에 약함. 기계 판독용 **전용 글꼴**도 등장 (광학식 OCR-A, 수표의 자기잉크식 E-13B)
- **사람이 설계한 특징 (1960–90년대)**: 방향 필터(Sobel, Gabor)의 획의 양, zoning (사실상 평균 풀링), 구멍·끝점·교차점 개수, 모멘트 → k-NN, SVM 등
- **한계**: 문자 체계마다 전문가가 특징을 새로 설계, 기울기·굵기에 약함, 버린 정보는 되살릴 수 없음

LeCun 등 (1998): **특징 추출기까지 데이터로 학습**하자. 학습된 첫 층 필터도 방향 에지 필터와 비슷해진다 — 어려웠던 것은 그 위의 **복잡한 조합**을 사람이 설계하는 일.

LeNet (1998)



LeNet-5의 구조 (LeCun 등, 1998). "$6@28 \times 28$"은 28×28 크기의 특징 맵 6장

- C = 합성곱, S = **서브샘플링** (2×2 합 $\times$ 학습 계수 $+$ 편향 $\rightarrow$ 활성화), F = 완전연결
- C5: 5×5 입력에 5×5 필터 120개 $\rightarrow 1 \times 1$ 출력 120개 (사실상 완전연결); 출력의 Gaussian connections $\rightarrow$ 지금은 Softmax
- SimpleConvNet과 거의 같은 구조. 지금과 다른 점: sigmoid 계열 (scaled tanh) $\rightarrow$ **ReLU**, 서브샘플링 $\rightarrow$ **최대 풀링**
- 거의 30년 전에 이미 지금의 CNN과 같은 기본 구조가 완성

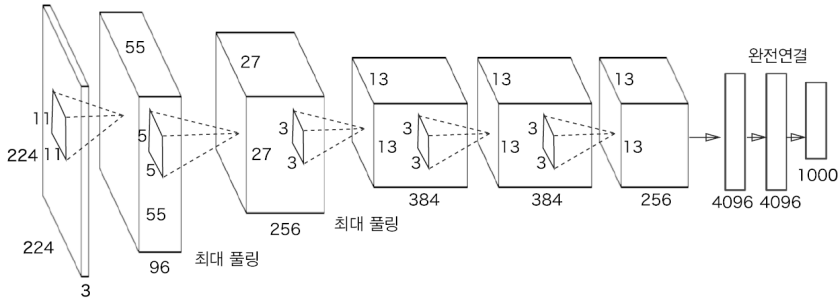
LeNet의 서브샘플링

서브샘플링 = “표본을 줄인다” (해상도를 낮춘다): LeNet의 풀링 계층. 특징 맵마다 겹치지 않는 2×2 영역 하나 $\rightarrow$ 출력 하나:

$$y = \sigma\big(w\,(x_1 + x_2 + x_3 + x_4) + b\big)$$

- 네 값을 더하고, 학습하는 계수 w를 곱하고 편향 b를 더한 뒤 활성화 σ (scaled tanh)
- 예: $\begin{pmatrix} 1 & 3 \\ 2 & 6 \end{pmatrix}$, $w = 0.25$, $b = 0 \rightarrow 0.25 \times 12 = 3 =$ 네 값의 **평균**
- 즉 **평균 풀링 (스트라이드 2)** + 학습하는 배율$\cdot$편향 + 활성화; $28 \times 28 \rightarrow 14 \times 14$
- 매개변수: 특징 맵마다 w, b 하나씩 (위치 공유) $\rightarrow$ S2 12개, S4 32개 (최대 풀링은 0개)
- 목적: 획의 위치가 조금 달라도 출력이 거의 같게 $\rightarrow$ 복합 세포, Neocognitron의 C층과 같은 역할

AlexNet (2012)



AlexNet의 구조 (Krizhevsky 등, 2012): 합성곱 계층 5개와 완전연결 계층 3개

- ILSVRC 2012 top-5 오류율 약 15% (2위 26%) → 딥러닝 열풍
- LeNet과 같은 큰 틀 + 변화: **ReLU**, **LRN** (국소적 정규화, 지금은 배치 정규화), **드롭아웃**, **최대 풀링**
- 매개변수 약 6000만 개 ⇐ **빅데이터** (ImageNet, 약 120만 장)와 **GPU** 덕분에 학습 가능
- 입력은 원 논문대로 224로 표기; 출력 55는 입력 227, 패딩 0일 때 공식과 맞는다 (연습문제 1)

AlexNet에는 지금까지 배운 부품이 모두 있다

AlexNet에는 지금까지 배운 부품이 모두 있다

- ReLU, 최대 풀링, 드롭아웃 (지난 주), 정규화 계층 (배치 정규화의 선배 격), GPU에서의 대규모 행렬곱 (im2col)
- 그 뒤의 CNN도 같은 부품을 **더 깊게** 쌓는 방향으로 발전
  - **VGG** (2014): 3×3 합성곱을 쌓아 $16 \sim 19$층 (완전연결 계층 포함)
  - **GoogLeNet** (2014): 여러 크기의 필터를 병렬로
  - **ResNet** (2015): 입력을 건너뛰어 더하는 skip connection으로 150층 이상

8

PyTorch로 구현하기

우리의 구현과 PyTorch

- 지난 시간의 PyTorch 완전연결 network와 같은 방법 → 바뀌는 것은 **계층의 이름**뿐 (`mnist_cnn_torch.py`)

우리의 구현 (`conv_layers.py`)	PyTorch
`Convolution(W, B, stride, pad)`	`nn.Conv2d(C, FN, kernel_size, stride, padding)`
`Pooling(2, 2)`	`nn.MaxPool2d(2)`
`Flatten()`	`nn.Flatten()`
필터 `W`: (FN, C, FH, FW)	`weight`: (FN, C, FH, FW) (같은 순서)

- 데이터는 $(N, 1, 28, 28)$ 모양의 텐서로 (`to_dataset`: `reshape(-1, 1, 28, 28)`, one-hot → 클래스 번호)
- 학습 루프는 지난 시간의 코드와 같다

PyTorch 모델 정의

```
model = nn.Sequential(
    nn.Conv2d(1, 30, kernel_size=5),      # (N, 1, 28, 28) -> (N, 30, 24, 24)
    nn.ReLU(),
    nn.MaxPool2d(2),                      # -> (N, 30, 12, 12)
    nn.Flatten(),                         # -> (N, 4320)
    nn.Linear(30 * 12 * 12, 100),
    nn.ReLU(),
    nn.Linear(100, 10),
).to(device)
```

- SimpleConvNet과 똑같은 구조: Conv → ReLU → Pool → Flatten → Affine → ReLU → Affine
- 주석의 shape 변화가 계층별 출력 shape 표와 같다; `.to(device)`로 CPU/GPU 선택

numpy 구현과 PyTorch의 비교

	검증 정확도					
	1 epoch	2 epoch	5 epoch	10 epoch	시험	epoch당
numpy (`conv_layers.py`)	95.4%	97.7%	98.6%	98.5%	98.5%	약 60초
PyTorch (CPU)	97.6%	98.4%	98.6%	99.1%	98.8%	약 12초

- 같은 구조, 같은 optimizer → 결과가 비슷 (초깃값·미니배치 순서가 달라 똑같지는 않음)
- 같은 CPU에서 PyTorch가 약 5배 빠름 (C++ 최적화 라이브러리); GPU가 있으면 `device="cuda"`만으로 더 빨라질 수 있음 (환경에 따라 다르니 실측)
- `nn.Conv2d`의 기본 초깃값: 균등분포 $(\pm 1/\sqrt{25})$

PyTorch로 학습한 첫 번째 층의 필터



PyTorch로 학습한 첫 번째 합성곱 계층의 필터 30개 (`model[0].weight`)

- **여러 방향의 에지** 모양 → 사람이 설계한 Sobel 필터와 닮은 필터를 데이터로부터 스스로 학습

9

정리

합성곱 신경망: 정리

합성곱 신경망: 정리

- CNN = 지금까지의 신경망 + **합성곱 계층** + **풀링 계층**
- 합성곱 계층은 이미지의 **형상을 유지**하며, 같은 필터를 모든 위치에 적용 (가중치 공유). 출력 크기 $OH = \lfloor \frac{H+2P-FH}{S} \rfloor + 1$
- 필터는 4차원 (FN, C, FH, FW), 데이터는 4차원 (N, C, H, W)
- 풀링 계층은 매개변수 없이 공간 크기를 줄이며, 작은 위치 변화에 강하다
- **im2col**을 쓰면 합성곱은 하나의 행렬곱 = Affine 계층. 역전파는 Affine의 역전파와 col2im
- 첫 층의 필터는 에지와 블롭에 반응하고, 층이 깊어질수록 더 추상적인 정보를 추출
- LeNet (1998)과 AlexNet (2012)이 대표적인 CNN; 빅데이터와 GPU가 딥러닝 발전의 큰 원동력

10

연습문제

연습문제 (1)

1. 출력 크기를 구하라. (a) 입력 $(32, 32)$, 필터 $(5, 5)$, $P = 2$, $S = 1$ (b) 입력 $(227, 227)$, 필터 $(11, 11)$, $P = 0$, $S = 4$ (AlexNet 첫 층) (c) $(28, 28)$에 2×2 최대 풀링 (스트라이드 2) 두 번
2. 입력 $\begin{pmatrix} 1 & 0 & 2 & 1 \\ 0 & 3 & 1 & 0 \\ 2 & 1 & 0 & 4 \\ 1 & 0 & 1 & 2 \end{pmatrix}$에 필터 $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ ($S = 1$, $P = 0$)를 적용한 결과를 손으로 계산하라. 같은 입력에 2×2 최대 풀링 ($S = 2$, $P = 0$)한 결과도 구하라.
3. RGB 이미지 ($C = 3$)에 3×3 필터 64개인 합성곱 계층의 매개변수 수는? 입력 $(3, 32, 32)$, $P = 1$일 때 같은 출력 $(64, 32, 32)$을 완전연결 계층으로 만들면 가중치는 몇 개?
4. 2번의 최대 풀링에서 출력 쪽 gradient가 $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$일 때, 입력 쪽 gradient (4×4)를 구하라.

연습문제 (2)

5. 길이 5인 1차원 입력 $(x_1, \ldots, x_5)$에 길이 3인 필터 (스트라이드 1): im2col 행렬 (3×3)을 쓰고, x_3이 몇 번 등장하는지 세어 보라. col2im에서 gradient를 **더해야** 하는 이유를 설명하라.

6. SimpleConvNet의 합성곱 계층에 패딩 2를 주도록 고쳐 보라. `conv_out`, `pool_out`, `flat`은 각각 얼마인가? `pad`를 넘기는 부분과 `flat`의 계산을 고친 뒤 `conv_basics.py`의 gradient 확인으로 검증하라.

7. `mnist_cnn_torch.py`를 고쳐 LeNet과 비슷한 network를 만들고 학습시켜라: Conv 5×5 6개 (패딩 2) $\to$ ReLU $\to$ MaxPool $\to$ Conv 5×5 16개 $\to$ ReLU $\to$ MaxPool $\to$ FC 120 $\to$ ReLU $\to$ FC 84 $\to$ ReLU $\to$ FC 10. 각 계층 뒤의 shape을 표로 정리하고, 매개변수 수와 검증 정확도를 SimpleConvNet과 비교하라.